

LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 6



Oleh :

M.YAZEM AGVA ROIZ

NIM 2511533003

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

KATA PENGANTAR

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah Pemrograman Dasar dengan Java. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2026

Penulis

DAFTAR ISI

BAB 1

PENDAHULUAN

1. Latar Belakang

Dalam pembuatan program diperlukannya sebuah struktur yang tepat untuk kasus tertentu, linked list merupakan salah satu struktur yang dapat diandalkan serta menjadi pondasi utama dalam banyak kasus.

Double linked list adalah salah satu struktur utama dalam pemrograman, memungkinkan menyimpan nilai yang dinamis serta memungkinkan untuk melakukan transversal 2 arah yang meningkatkan nilai fungsionalitasnya dalam banyak kasus.

2. Tujuan

Tujuan dari praktikum ini adalah mempelajari linked list dengan cara membuat codenya untuk mendapatkan pemahaman yang dalam serta melakukan validasi atas code yang dibuat untuk mendapatkan pemahaman yang benar.

3. Manfaat

Manfaat melakukan praktikum ini adalah mendapatkan pemahaman yang dalam dan benar serta melatih dalam menulis kode berbasis code linked list dengan benar dan tepat.

BAB 2

PEMBAHASAN

A. Teori

Double Linked List merupakan struktur data yang memiliki 2 pointer, berbeda dengan single linked list yang hanya mengarah pada satu pointer saja yaitu node selanjutnya (next), pada double linked list ini Node memiliki 2 pointer yaitu menunjuk ke Node setelahnya (next) dan ke Node sebelumnya (prev).

Dengan pendekatan ini, membuat double linked list bukan hanya lebih leluasa dalam ukuran yang dinamis, namun memiliki kebebasan yang lebih karena memiliki penelusuran 2 arah, keuntungan ini membuat transversal semakin luas, penghapusan Node semakin mudah karena pada satu node mengetahui Node apa saja yang berada setelah dan sebelumnya.

B. Code

a. NodeDLL_2511533003.java

```
package pekan6_2511533003;
public class NodeDLL_2511533003 {
    // Node definition
    int data_3003; // data
    NodeDLL_2511533003 next_3003; // pointer ke next node
    NodeDLL_2511533003 prev_3003; // pointer ke previous node

    // konstruktor
    public NodeDLL_2511533003(int data_3003) {
        this.data_3003 = data_3003;
        this.next_3003 = null;
        this.prev_3003 = null;
    }
}
```

Code diatas merupakan code ADT yang memungkinkan pembuatan double linked list, ADT ini memiliki 3 properti yaitu:

1. data_3003, berfungsi untuk menyimpan value pada node.
2. next_3003, berfungsi untuk menunjuk ke node setelahnya.
3. Prev_3003, berfungsi untuk menunjuk kepada node sebelumnya.

Code ADT ini akan berfungsi sebagai Node dan properti pada ADT ini dapat menjadi penghubung antar node untuk tujuan membangun Double Linked List.

b. insertDLL_2511533003.java

```
package pekan6_2511533003;

public class insertDLL_2511533003 {

    static NodeDLL_2511533003 insertBegin_3003(
        NodeDLL_2511533003 head_3003,
        int data_3003
    ) {
        // buat node baru
        NodeDLL_2511533003 new_node_3003 = new
NodeDLL_2511533003(data_3003);
        new_node_3003.next_3003 = head_3003;

        if (head_3003 != null) {
            head_3003.prev_3003 = new_node_3003;
        }

        return new_node_3003;
    }

    static NodeDLL_2511533003 insertEnd_3003(
        NodeDLL_2511533003 head_3003,
        int newData_3003
    ) {
        NodeDLL_2511533003 newNode_3003 = new
NodeDLL_2511533003(newData_3003);

        if (head_3003 == null) {
            head_3003 = newNode_3003;
        } else {
            NodeDLL_2511533003 curr_3003 = head_3003;
            while (curr_3003.next_3003 != null) {
                curr_3003 = curr_3003.next_3003;
            }
            curr_3003.next_3003 = newNode_3003;
        }
    }
}
```

```

        newNode_3003.prev_3003 = curr_3003;
    }

    return head_3003;
}

public static NodeDLL_2511533003 insertAtPosition_3003(
    NodeDLL_2511533003 head_3003,
    int pos_3003,
    int new_data_3003
) {
    NodeDLL_2511533003 new_node_3003 = new NodeDLL_2511533003(
        new_data_3003
    );
    if (pos_3003 == 1) {
        new_node_3003.next_3003 = head_3003;
        if (head_3003 != null) {
            head_3003.prev_3003 = new_node_3003;
        }
        head_3003 = new_node_3003;
        return head_3003;
    }

    NodeDLL_2511533003 curr_3003 = head_3003;
    for (
        int i_3003 = 1;
        i_3003 < pos_3003 - 1 && curr_3003 != null;
        ++i_3003
    ) {
        curr_3003 = curr_3003.next_3003;
    }

    if (curr_3003 == null) {
        System.out.println("Posisi Tidak Ada");
        return head_3003;
    }

    new_node_3003.prev_3003 = curr_3003;
    new_node_3003.next_3003 = curr_3003.next_3003;
    curr_3003.next_3003 = new_node_3003;
    if (new_node_3003.next_3003 != null) {
        new_node_3003.next_3003.prev_3003 = new_node_3003;
    }

    return head_3003;
}

public static void printList_3003(NodeDLL_2511533003 head_3003) {
    NodeDLL_2511533003 curr_3003 = head_3003;

```

```

        while (curr_3003 != null) {
            System.out.print(curr_3003.data_3003 + " <-> ");
            curr_3003 = curr_3003.next_3003;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeDLL_2511533003 head_3003 = new NodeDLL_2511533003(2);
        head_3003.next_3003 = new NodeDLL_2511533003(3);
        head_3003.next_3003.prev_3003 = head_3003;
        head_3003.next_3003.next_3003 = new NodeDLL_2511533003(5);
        head_3003.next_3003.next_3003.prev_3003 = head_3003.next_3003;

        System.out.print("DLL Awal: ");
        printList_3003(head_3003);

        head_3003 = insertBegin_3003(head_3003, 1);

        System.out.println("Simpul 1 ditambah diawal: ");
        printList_3003(head_3003);

        System.out.print("Simpul 6 ditambah diakhir: ");
        int data_3003 = 6;
        head_3003 = insertEnd_3003(head_3003, data_3003);

        printList_3003(head_3003);

        System.out.print("tambah Node 4 posisi 4: ");

        int data2_3003 = 4;
        int pos_3003 = 4;
        head_3003 = insertAtPosition_3003(head_3003, pos_3003, data2_3003);
        printList_3003(head_3003);
    }
}

```

Code diatas merupakan Code driver, code tersebut akan menggunakan ADT Node yang telah dibuat untuk membangun suatu ADT.

Tersedia method yang berfungsi untuk menambahkan Node ke dalam linked list berdasarkan posisi, method yang tersedia antara lain:

1. insertBegin_3003

Method yang berfungsi untuk menambahkan Node baru ke depan dari linked list.

2. insertEnd_3003

Method yang berfungsi untuk menambahkan Node baru ke belakang dari linked list.

3. insertAtPosition_3003

Method yang berfungsi untuk memasukkan Node ke dalam linked list berdasarkan posisi yang ditetapkan melalui parameter.

4. printList_3003

Method yang berfungsi untuk menampilkan linked list dengan cara menelusuri Node.

c. PenelusuranDLL_2511533003.java

```
package pekan6_2511533003;
public class PenelusuranDLL_2511533003 {
    static void forwalsTraversal_3003(NodeDLL_2511533003 head_3003) {
        NodeDLL_2511533003 curr_3003 = head_3003;
        while (curr_3003 != null) {
            System.out.print(curr_3003.data_3003 + " <-> ");
            curr_3003 = curr_3003.next_3003;
        }
        System.out.println();
    }

    static void backwardTransversal(NodeDLL_2511533003 tail_3003) {
        NodeDLL_2511533003 curr_3003 = tail_3003;
        while (curr_3003 != null) {
            System.out.print(curr_3003.data_3003 + " <-> ");
            curr_3003 = curr_3003.prev_3003;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeDLL_2511533003 head_3003 = new NodeDLL_2511533003(1);
        NodeDLL_2511533003 second_3003 = new NodeDLL_2511533003(2);
```

```

NodeDLL_2511533003 third_3003 = new NodeDLL_2511533003(3);

head_3003.next_3003 = second_3003;
second_3003.prev_3003 = head_3003;
second_3003.next_3003 = third_3003;
third_3003.prev_3003 = second_3003;

System.out.println("Penelusuran maju:");
forwaldTraversal_3003(head_3003);
System.out.println("Penelusuran mundur:");
backwardTransversal(third_3003);
    }
}

```

Code diatas merupakan Code driver yang memiliki tujuan untuk menelusuri linked list yang telah terbangun, karena double linked list memungkinkan 2 arah penelusuran maka ada 2 method yang tercipta, antara lain:

1. forwaldTraversal_3003

Berfungsi untuk menelusuri linked list dari depan ke belakang.

2. backwardTransversal_3003

Berfungsi untuk menelusuri linked list dari belakang ke depan.

d. HapusDLL_2511533003.java

```

package pekan6_2511533003;

public class HapusDLL_2511533003 {

    public static NodeDLL_2511533003 delHead_3003(
        NodeDLL_2511533003 head_3003
    ) {
        if (head_3003 == null) {
            return null;
        }

        NodeDLL_2511533003 temp_3003 = head_3003;
        head_3003 = head_3003.next_3003;
        if (head_3003 != null) {

```

```

        head_3003.prev_3003 = null;
    }
    return head_3003;
}

public static NodeDLL_2511533003 delLast_3003(
    NodeDLL_2511533003 head_3003
) {
    if (head_3003 == null) {
        return null;
    }

    if (head_3003.next_3003 == null) {
        return null;
    }

    NodeDLL_2511533003 curr_3003 = head_3003;

    while (curr_3003.next_3003 != null) {
        curr_3003 = curr_3003.next_3003;
    }

    if (curr_3003.prev_3003 != null) {
        curr_3003.prev_3003.next_3003 = null;
    }

    return head_3003;
}

public static NodeDLL_2511533003 delPos_3003(
    NodeDLL_2511533003 head_3003,
    int pos_3003
) {
    if (head_3003 == null) {
        return head_3003;
    }

    NodeDLL_2511533003 curr_3003 = head_3003;
    for (int i_3003 = 1; curr_3003 != null && i_3003 < pos_3003; ++i_3003) {
        curr_3003 = curr_3003.next_3003;
    }

    if (curr_3003 == null) {
        return head_3003;
    }

    if (curr_3003.prev_3003 != null) {
        curr_3003.prev_3003.next_3003 = curr_3003.next_3003;
    }
}

```

```

        if (curr_3003.next_3003 != null) {
            curr_3003.next_3003.prev_3003 = curr_3003.prev_3003;
        }

        if (head_3003 == curr_3003) {
            head_3003 = curr_3003.next_3003;
        }

        return head_3003;
    }

    public static void printList(NodeDLL_2511533003 head_3003) {
        NodeDLL_2511533003 curr_3003 = head_3003;
        while (curr_3003 != null) {
            System.out.print(curr_3003.data_3003 + " <-> ");
            curr_3003 = curr_3003.next_3003;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeDLL_2511533003 head_3003 = new NodeDLL_2511533003(1);
        head_3003.next_3003 = new NodeDLL_2511533003(2);
        head_3003.next_3003.prev_3003 = head_3003;
        head_3003.next_3003.next_3003 = new NodeDLL_2511533003(3);
        head_3003.next_3003.next_3003.prev_3003 = head_3003.next_3003;
        head_3003.next_3003.next_3003.next_3003 = new NodeDLL_2511533003(4);
        head_3003.next_3003.next_3003.next_3003.prev_3003 =
            head_3003.next_3003.next_3003;
        head_3003.next_3003.next_3003.next_3003.next_3003 =
            new NodeDLL_2511533003(5);
        head_3003.next_3003.next_3003.next_3003.next_3003.prev_3003 =
            head_3003.next_3003.next_3003.next_3003;

        System.out.print("DLL Awal: ");
        printList(head_3003);

        System.out.print("Setelah Head Dihapus");
        head_3003 = delHead_3003(head_3003);
        printList(head_3003);

        System.out.print("Setelah node terakhir dihapus");
        head_3003 = delLast_3003(head_3003);
        printList(head_3003);

        System.out.print("Menghapus Node Kedua");
        head_3003 = delPos_3003(head_3003, 2);
        printList(head_3003);
    }

```

```
}  
}
```

Code diatas merupakan code driver yang menggunakan ADT Node yang telah dibuat untuk membangun double linked list, code ini berfungsi untuk melakukan penghapusan berdasarkan lokasi, method yang tersedia diantaranya:

1. delHead_3003

Method yang Berfungsi untuk menghilangkan Node terdepan dari linked list.

2. delLast_3003

Method yang Berfungsi untuk menghilangkan Node paling belakang dari linked list.

3. delPos_3003

Method yang berfungsi untuk menghapus Node berdasarkan posisi yang telah ditetapkan dari parameter.

4. printList

Berfungsi untuk menampilkan Node pada linked list.

C. Tugas

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

- a. Lagu_2511533003.java
package pekan6_2511533003;

```

public class Lagu_2511533003 {
    String judul_3003;
    String penyanyi_3003;
    Lagu_2511533003 next_3003;
    Lagu_2511533003 prev_3003;

    // konstruktor
    public Lagu_2511533003(String judul_3003, String penyanyi_3003) {
        this.judul_3003 = judul_3003;
        this.penyanyi_3003 = penyanyi_3003;
    }

    // getter
    public String getJudul_3003() {
        return this.judul_3003;
    }

    public String getPenyanyi_3003() {
        return this.penyanyi_3003;
    }

    public Lagu_2511533003 getNext_3003() {
        return this.next_3003;
    }

    public Lagu_2511533003 getPrev_3003() {
        return this.prev_3003;
    }

    // setter
    public void setJudul_3003(String judu_3003l) {
        this.judul_3003 = judu_3003l;
    }

    public void setPenyanyi_3003(String penyany_3003i) {
        this.penyanyi_3003 = penyany_3003i;
    }

    public void setNext_3003(Lagu_2511533003 nex_3003t) {
        this.next_3003 = nex_3003t;
    }

    public void setPrev_3003(Lagu_2511533003 prev) {
        this.prev_3003 = prev;
    }
}

```

Code diatas merupakan Code ADT yang akan digunakan untuk membangun Double Linked List, memiliki properti antara lain:

1. judul_3003, berguna untuk menyimpan data judul lagu.
2. Penyanyi_3003, berguna untuk menyimpan penyanyi dari suatu lagu.
3. next_3003, berfungsi untuk mengarah kepada Node atau lagu selanjutnya.
4. prev_3003, berfungsi untuk mengarah kepada Node atau lagu sebelumnya.

ADT ini pun juga terdapat method yang berfungsi untuk konstruksi ,getter serta setter.

b. Musik_2511533003.java

```
package pekan6_2511533003;
import java.util.Scanner;
public class Musik_2511533003 {

    public static Lagu_2511533003 tambahLagu_3003(
        Lagu_2511533003 head_3003,
        Lagu_2511533003 tail_3003
    ) {
        Scanner sc_3003 = new Scanner(System.in);

        // input
        System.out.print("judul: ");
        String judul_3003 = sc_3003.nextLine();

        System.out.print("Penyanyi: ");
        String penyanyi_3003 = sc_3003.nextLine();

        // Node
        Lagu_2511533003 lagu_3003 = new Lagu_2511533003(
            judul_3003,
            penyanyi_3003
        );

        return lagu_3003;
    }

    public static void tampilMaju_3003(Lagu_2511533003 head_3003) {
```

```

Lagu_2511533003 curr_3003 = head_3003;
if (curr_3003 == null) {
    System.out.println("Linked List Kosong!");
}

// menampilkan secara iterative node per node dari head
while (true) {
    if (curr_3003 == null) {
        break;
    }

    System.out.println(
        "judul musik: " +
        curr_3003.getJudul_3003() +
        ", penyayi: " +
        curr_3003.getPenyanyi_3003()
    );

    curr_3003 = curr_3003.next_3003;
}

public static Lagu_2511533003 hapusLaguAwal_3003(
    Lagu_2511533003 head_3003
) {
    if (head_3003 == null) {
        System.out.println("Linked List Kosong!");
        return null;
    }

    return head_3003.next_3003;
}

public static void tampilMundur_3003(Lagu_2511533003 tail_3003) {
    Lagu_2511533003 curr_3003 = tail_3003;
    if (curr_3003 == null) {
        System.out.println("Linked List Kosong!");
    }

    // menampilkan secara iterative node per node dari tail
    while (true) {
        if (curr_3003 == null) {
            break;
        }

        System.out.println(
            "judul musik: " +
            curr_3003.getJudul_3003() +
            ", penyayi: " +

```



```

        curr_3003.getPenyanyi_3003()
    );

    curr_3003 = curr_3003.prev_3003;
}
}

public static void cariLagu(Lagu_2511533003 head_3003) {
    Lagu_2511533003 curr_3003 = head_3003;
    if (curr_3003 == null) {
        System.out.println("Linked List Kosong!");
        return;
    }

    Scanner sc_3003 = new Scanner(System.in);
    System.out.print("masukkan judul yang ingin dicari: ");
    // input
    String cariJudul_3003 = sc_3003.nextLine();

    // pencarian dimulai dari head
    while (true) {
        if (curr_3003 == null) {
            System.out.println(
                "Lagu dengan judul " + cariJudul_3003 + " tidak ditemukan"
            );
            break;
        }

        if (curr_3003.getJudul_3003().equalsIgnoreCase(cariJudul_3003)) {
            System.out.println(
                "Lagu dengan judul " + cariJudul_3003 + " ditemukan"
            );
            break;
        }

        curr_3003 = curr_3003.next_3003;
    }
}

public static void main(String[] args) {
    Scanner sc_3003 = new Scanner(System.in);
    Lagu_2511533003 head_3003 = null;
    Lagu_2511533003 tail_3003 = null;

    while (true) {
        System.out.println("\n=== Playlist Musik NIM: 2411531005 === ");
        System.out.println("1. Tambah Lagu ");
        System.out.println("2. Hapus Lagu Pertama ");
        System.out.println("3. Lihat Playlist (Maju) ");
    }
}

```

```

        System.out.println("4. Lihat Playlist (Mundur) ");
        System.out.println("5. Cari Lagu ");
        System.out.println("6. Keluar \n");
        System.out.print("pilihan: ");

        int option_3003 = sc_3003.nextInt();

        if (option_3003 == 1) {
            Lagu_2511533003 lagu_3003 = tambahLagu_3003(
                head_3003,
                tail_3003
            );
            if (head_3003 == null && tail_3003 == null) {
                head_3003 = lagu_3003;
                tail_3003 = lagu_3003;
            } else {
                lagu_3003.prev_3003 = tail_3003;
                tail_3003.next_3003 = lagu_3003;
                tail_3003 = lagu_3003;
            }

            System.out.println("berhasil ditambahkan! ");
        } else if (option_3003 == 2) {
            head_3003 = hapusLaguAwal_3003(head_3003);
            if (head_3003 == null) {
                head_3003 = null;
                tail_3003 = null;
                continue;
            }
            head_3003.prev_3003 = null;
            System.out.println("berhasil di hapus");
        } else if (option_3003 == 3) {
            tampilMaju_3003(head_3003);
        } else if (option_3003 == 4) {
            tampilMundur_3003(tail_3003);
        } else if (option_3003 == 5) {
            cariLagu(head_3003);
        } else if (option_3003 == 6) {
            break;
        } else {
            System.out.println("pilihan tidak ada di opsi");
        }
    }

    sc_3003.close();
}
}

```

Code diatas merupakan code driver yang akan menggunakan ADT lagu yang telah dibuat sebelumnya dengan tujuan menyimpan lagu lagu yang telah dibuat lalu disimpan ke dalam bentuk Double Linked List.

Saat memulai program, maka program akan meminta user untuk memilih opsi yang telah disediakan.

```
=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: █
```

Untuk dapat menambahkan lagu, pilih opsi 1. Program akan meminta user untuk memasukkan judul serta penyanyi, program akan secara otomatis menyambungkan lagu tersebut ke akhir dari Double Linked List.

```
=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 1
judul: a
Penyanyi: a
berhasil ditambahkan!

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 1
judul: b
Penyanyi: b
berhasil ditambahkan!

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 1
judul: c
Penyanyi: c
berhasil ditambahkan!

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: █
```

Dari kode diatas, user membuat 3 lagu yang telah dimasukkan ke dalam double linked list. Setelah lagu dimasukkan, User dapat melihat list lagu apa saja menggunakan opsi 3 untuk melihat list lagu dari depan atau opsi 4 untuk melihat list lagu dari belakang.

```
=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 3
judul musik: a, penyayi: a
judul musik: b, penyayi: b
judul musik: c, penyayi: c

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 4
judul musik: c, penyayi: c
judul musik: b, penyayi: b
judul musik: a, penyayi: a
```

Setelah melihat list lagu apa saja, use dapat mencari lagu berdasarkan judulnya dengan opsi 5.

```

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 5
masukkan judul yang ingin dicari: a
Lagu dengan judul a ditemukan

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 5
masukkan judul yang ingin dicari: z
Lagu dengan judul z tidak ditemukan

```

Jika lagu ditemukan, maka program akan memberikan output berupa judul ditemukan sebaliknya pada judul lagu yang tidak ada. Program pun memiliki opsi untuk menghapus lagu pertama melalui opsi 2.

```

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 2
berhasil di hapus

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 3
judul musik: b, penyayi: b
judul musik: c, penyayi: c

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 4
judul musik: c, penyayi: c
judul musik: b, penyayi: b

```

Jika terdapat Double kasus linked list kosong, maka program akan memberikan notifikasi untuk itu.

```
=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 2
Linked List Kosong!

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 3
Linked List Kosong!
```

```
pilihan: 4
Linked List Kosong!

=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

pilihan: 5
Linked List Kosong!
```

BAB 3

PENUTUPAN

a. Kesimpulan

Dengan penulisan code Double linked list ini dapat dipahami adalah Double linked list merupakan struktur data yang sangatlah berguna untuk kasus yang memerlukan relasi antar data dengan mempertahankan urutan, tanpa adanya size tetap membuat struktur ini sangatlah fleksibel dan berguna dalam kasus yang memiliki size yang tidak pasti serta memerlukan penelusuran 2 arah.

b. Saran

Laporan praktikum yang telah penulis tuliskan, masih memiliki kekurangan. Penulis sendiri pun juga membuka saran dan kritikan untuk meningkatkan kualitas laporan pratikum ini.

DAFTAR PUSTAKA

Oracle. *Class LinkedList*. Java Platform SE 8 Documentation. Diakses dari https://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html?utm_source=chatgpt.com pada 13 Mei 2026.